UNIT 16 90 min

NumPy: Arrays and Fast Math

Arrays with a fixed type and shape, math without loops, indexing and masks, broadcasting, and the matrix product.

By the end of this unit you will be able to

- Create arrays and explain how they differ from Python lists
- Read an array's `shape`, `dtype`, and `ndim`
- Use vectorized math instead of loops, and explain why it is faster
- Index, slice, and select with boolean masks in 1-D and 2-D arrays
- Aggregate along an axis and predict the shape of the result
- Use broadcasting and the `@` matrix product

Why NumPy

NumPy stores numbers in a compact block of memory, all with the same type, and runs its math in optimized code. An operation on a whole array happens in one step, with no Python loop. This is the foundation for pandas, scikit-learn, and PyTorch: all of them use NumPy-style arrays underneath.

import_numpy.py

```
import numpy as np

a = np.array([1, 2, 3, 4])
print(a)
print(a * 10)
print(a.shape, a.dtype)
```

> **Note**
> The convention is `import numpy as np`. Nearly every tutorial and codebase uses `np`.

Arrays versus lists

	List	NumPy array
Element types	Can mix types	One type for all items (`dtype`)
`[1, 2] * 2`	`[1, 2, 1, 2]` (repeats)	`[2 4]` (multiplies each item)
`+` between two	Joins them	Adds item by item
Math on every item	Needs a loop or comprehension	Written directly
Speed on large data	Slower	Much faster

> **Watch out**
> `[1, 2] * 2` repeats the list. `np.array([1, 2]) * 2` multiplies each number. Mixing these up is the most common NumPy beginner error.

Creating arrays

creating.py

```
import numpy as np

print(np.zeros(3))
print(np.ones((2, 3)))
print(np.arange(0, 10, 2))
print(np.linspace(0, 1, 5))
print(np.eye(2))
```

`np.arange(start, stop, step)` works like `range` but returns an array. `np.linspace(start, stop, count)` gives evenly spaced values and includes `stop`. The shape is a tuple: `(2, 3)` means two rows, three columns.

Shape, dtype, and reshape

shape.py

```
import numpy as np

a = np.arange(6)
print(a.shape)
grid = a.reshape(2, 3)
print(grid)
print(grid.shape, grid.ndim)
print(np.array([1.5, 2]).dtype)
```

`reshape` changes the shape without changing the data. The new shape must hold the same number of items. Use `-1` to let NumPy calculate one dimension for you: `a.reshape(-1, 2)` gives three rows here.

Indexing and slicing

For a 2-D array, `grid[row, col]` picks one element. A colon selects a whole range. `grid[:, 1]` means "all rows, column 1".

indexing.py

```
import numpy as np

grid = np.arange(1, 13).reshape(3, 4)
print(grid)
print(grid[0, 2])
print(grid[1])
print(grid[:, 1])
print(grid[1:, :2])
```

Boolean masks

Comparing an array with a value gives a **boolean mask**: an array of True and False with the same shape. Use the mask inside square brackets to keep only the True positions. This is how you filter

data without a loop.

masks.py

```
import numpy as np

scores = np.array([72, 95, 88, 61, 90])
passing = scores >= 75
print(passing)
print(scores[passing])
print(np.count_nonzero(passing))
scores[scores < 65] = 65
print(scores)
```

Tip

Combine masks with `&` (and), `|` (or), and `~` (not). Put each comparison in parentheses: `(a > 1) & (a < 5)`. Plain `and` does not work on arrays.

Vectorized math and aggregation

Operations like `+`, `*`, `np.sqrt`, and `np.exp` apply to every element. Aggregations like `sum`, `mean`, `std`, `min`, and `max` reduce an array. With `axis=0` they reduce down the rows (one result per column). With `axis=1` they reduce across the columns (one result per row).

aggregate.py

```
import numpy as np

grid = np.array([[1, 2, 3],
                 [4, 5, 6]])
print(grid.sum())
print(grid.sum(axis=0))
print(grid.sum(axis=1))
print(grid.mean(axis=1))
print(np.sqrt(np.array([4, 9, 16])))
```

Broadcasting

When two arrays have different shapes, NumPy **broadcasts** the smaller one: it stretches it across the larger one without copying data. A single number, or a row that matches the last dimension, can be combined with a whole table.

broadcast.py

```
import numpy as np

table = np.array([[10, 20, 30],
                  [40, 50, 60]])
bonus = np.array([1, 2, 3])
print(table + bonus)
print(table * 0.5)
```

Trace: what broadcasting does TRACE

```
import numpy as np

table = np.array([[10, 20], [30, 40]])
bonus = np.array([1, 2])
result = table + bonus
print(result)
```

Step	Line run	Variables after the line	Output so far
1	3	`np = module, table = 2 x 2 array`	—
2	4	`np = module, table = 2 x 2 array, bonus = shape (2,)`	—
3	5	`np = module, table = 2 x 2 array, bonus = shape (2,), result = [[11 22] [31 42]]`	—
4	6	`np = module, table = 2 x 2 array, bonus = shape (2,), result = [[11 22] [31 42]]`	`[[11 22] [31 42]]`

Matrix product

The `@` operator performs matrix multiplication. The inner sizes must match: a (2, 3) array times a (3, 4) array gives a (2, 4) array. `*` is different: it multiplies element by element.

matmul.py

```
import numpy as np

A = np.array([[1, 2], [3, 4]])
B = np.array([[0, 1], [1, 0]])
print(A * B)
print(A @ B)
```

Random numbers with a seed

For reproducible results, create a generator with a seed. The same seed gives the same numbers every run, which matters for tests and for teaching.

rng.py

```
import numpy as np

rng = np.random.default_rng(42)
print(rng.integers(1, 7, size=5))
print(rng.random(2).round(3))
```

Predict the output THINK FIRST

Pay attention to shape, and to `*` versus `@`.

```
import numpy as np
print(np.array([1, 2]) * 2)
```

Answer: `[2 4]`

```
import numpy as np
print(np.zeros((2, 3)).shape)
```

Answer: `(2, 3)`

```
import numpy as np
print(np.arange(5)[::2])
```

Answer: `[0 2 4]`

```
import numpy as np
g = np.arange(6).reshape(2, 3)
print(g.sum(axis=0))
```

Answer: `[3 5 7]`
Columns: 0+3, 1+4, 2+5.

```
import numpy as np
print(np.array([1, 5, 9]) > 4)
```

Answer: `[False True True]`

```
import numpy as np
print(np.arange(6).reshape(2, 3).reshape(-1).shape)
```

Answer: `(6,)`

Practice

▼ **Exercise 1 — Temperatures**

Given `celsius = np.array([0, 20, 37, 100])`, compute an array of Fahrenheit values with one expression, and print the values that are above 80°F using a mask.

▼ **Exercise 2 — Column means**

Create a 4-by-3 array with `np.arange(12).reshape(4, 3)`. Print the mean of each column, then the total of all values.

Discussion

1. A list comprehension and a NumPy expression can give the same answer. What does each make easier to read, and what does each make easier to speed up?
2. Why does NumPy require one dtype for all items? What would be lost if it allowed mixed types?
3. Broadcasting can silently combine shapes you did not intend. How would you check that a result has the shape you expect?

Key terms

ndarray

NumPy's n-dimensional array: a block of same-typed values with a shape.

shape

A tuple giving the size of each dimension, such as `(2, 3)` for two rows and three columns.

dtype

The data type of every element in an array, such as `int64` or `float64`.

Vectorization

Writing an operation on a whole array at once instead of looping over its elements.

Boolean mask

An array of True and False values, used to select elements that meet a condition.

Axis

The dimension an aggregation runs along. `axis=0` runs down rows; `axis=1` runs across columns.

Broadcasting

The rules NumPy uses to combine arrays of different shapes by stretching the smaller one.

Reshape

Changing an array's shape while keeping the same data and the same number of elements.

Matrix product

The `@` operator, which multiplies two arrays using rows of the first and columns of the second.

See all terms in the glossary →